SELENIUM WEBDRIVER PRACTICALS 6–10

Python language • Visual Studio Code • Windows

Included in this guide

• Practical 6 — Install Selenium WebDriver and configure Python environment

• Practical 7 — Automate Google Search

• Practical 8 — Automate a login form using OrangeHRM demo

• Practical 9 — Use ID, Name, XPath and CSS Selector locators

• Practical 10 — Handle alerts, frames/iframes and multiple windows

Note: These are Python versions for VS Code. Java, Eclipse and Maven are not required for these versions. Internet access is useful for installing packages, browser drivers and loading demo websites.

Step-by-step setup, commands, complete code and expected results.

Before starting: Install and prepare VS Code

1. Install the software

• Install Python 3 from python.org/downloads. During setup, tick **Add Python to PATH**.

• Install Visual Studio Code from code.visualstudio.com/download.

• Install Google Chrome from google.com/chrome.

• In VS Code, press **Ctrl + Shift + X**, search for **Python** by Microsoft, and install the extension.

2. Create your project folder

• Create a folder named Selenium_Practicals on your Desktop.

• Open VS Code → File → Open Folder → choose Selenium_Practicals.

• Open Terminal → New Terminal.

• Create one Python file for each practical using the filenames shown below.

3. Verify Python and pip

```
python --version
python -m pip --version
```

If python is not recognised, try:

```
py --version
py -m pip --version
```

4. Install Selenium

```
python -m pip install selenium
python -m pip show selenium
```

Selenium Manager can usually find or download a compatible ChromeDriver automatically when the script starts. A separate manual ChromeDriver installation is usually not needed. Keep internet access available for first-time driver setup.

Suggested folder structure

```
Selenium_Practicals/
■■■ practical6_setup.py
■■■ practical7_google_search.py
■■■ practical8_login.py
■■■ practical9_locators.py
■■■ practical10_alerts_frames_windows.py
```

Practical 6 — Install Selenium WebDriver and Configure Environment

Aim: To install Selenium WebDriver and configure the development environment using Python in VS Code.

Procedure

• Install Python, VS Code, the VS Code Python extension and Google Chrome.

• Open the project folder in VS Code.

• Run python -m pip install selenium in the terminal.

• Create practical6_setup.py and enter the program below.

• Run the file using python practical6_setup.py.

Program: practical6_setup.py

```
from selenium import webdriver

# Launch Chrome browser
driver = webdriver.Chrome()

try:
    # Open Google
    driver.get("https://www.google.com")

    # Print the page title
    print("Page Title:", driver.title)

finally:
    # Close the browser session
    driver.quit()
```

Run command

```
python practical6_setup.py
```

Expected output

```
Page Title: Google
```

Result: Selenium was installed and used to launch Chrome, open Google, print the page title, and close the browser.

Note: If Chrome opens and closes quickly, that is expected. If setup fails, check Chrome installation, internet access, and that Selenium was installed for the Python interpreter selected in VS Code.

Practical 7 — Automate Google Search Using Selenium WebDriver

Aim: To automate a Google search, enter a keyword, submit the search, and verify the page title and current URL.

Algorithm

• Launch Chrome and open Google.

• Locate the search box using the Name locator.

• Enter the keyword Selenium and submit the search.

• Read the page title and current URL.

• Verify that both contain the keyword, then close the browser.

Program: practical7_google_search.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

try:
    driver.maximize_window()
    driver.get("https://www.google.com")

    search_box = WebDriverWait(driver, 10).until(
        EC.visibility_of_element_located((By.NAME, "q"))
    )
    search_box.send_keys("Selenium")
    search_box.submit()

    WebDriverWait(driver, 15).until(
        lambda d: "selenium" in d.current_url.lower()
    )

    page_title = driver.title
    current_url = driver.current_url
    print("Page Title:", page_title)

    if "selenium" in page_title.lower():
        print("PASS: Page title is verified.")
    else:
        print("FAIL: Page title verification failed.")

    if "selenium" in current_url.lower():
        print("PASS: Search URL contains the keyword.")
    else:
        print("FAIL: Search URL verification failed.")

finally:
    driver.quit()
```

Run command

```
python practical7_google_search.py
```

Expected output (may vary)

```
Page Title: Selenium - Google Search
PASS: Page title is verified.
PASS: Search URL contains the keyword.
```

Observation: Google may display a CAPTCHA or unusual-traffic page during automated access. If that happens, title or URL verification can fail even when the Selenium setup is correct.

Result: A Selenium script was used to automate a Google search and verify the resulting page information.

Practical 8 — Automate Login Form Using Selenium WebDriver

Aim: To enter a username and password on a demo login form, click Login, and verify whether the dashboard opens.

Demo website: OrangeHRM Demo Login

Demo credentials from the supplied practical: Username: Admin | Password: admin123. These public demo credentials may change or stop working.

Program: practical8_login.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 20)

try:
    driver.maximize_window()
    print("Chrome browser launched.")

    driver.get(
        "https://opensource-demo.orangehrmlive.com/web/index.php/auth/login"
    )
    print("OrangeHRM login page opened.")

    username = wait.until(
        EC.visibility_of_element_located((By.NAME, "username"))
    )
    username.send_keys("Admin")
    print("Username entered successfully.")

    password = driver.find_element(By.NAME, "password")
    password.send_keys("admin123")
    print("Password entered successfully.")

    driver.find_element(
        By.XPATH, "//button[@type='submit']"
    ).click()
    print("Login button clicked.")

    try:
        wait.until(EC.url_contains("dashboard"))
        print("PASS: Login Successful.")
    except Exception:
        print("FAIL: Login Failed or dashboard did not load.")

    print("Page Title:", driver.title)
    print("Current URL:", driver.current_url)

finally:
    driver.quit()
    print("Browser closed.")
```

Run command

```
python practical8_login.py
```

Expected output (if demo login succeeds)

```
Chrome browser launched.
OrangeHRM login page opened.
Username entered successfully.
Password entered successfully.
Login button clicked.
PASS: Login Successful.
Page Title: OrangeHRM
Browser closed.
```

Result: The login form was automated and the result checked using the current URL. If the website changes, rejects the demo credentials or does not load, the test may report failure.

Practical 9 — Implement Selenium Scripts Using Different Locators

Aim: To identify and interact with web elements using ID, Name, XPath and CSS Selector locators.

Locator summary

Locator	Python example	Purpose
ID	By.ID, "app"	Application container
Name	By.NAME, "username"	Username field
Name	By.NAME, "password"	Password field
XPath	By.XPATH, //button[@type=submit]	Login button
CSS Selector	h6.oxd-topbar-header-breadcrumb-module	Dashboard heading

Program: practical9_locators.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 20)

try:
    driver.maximize_window()
    driver.get(
        "https://opensource-demo.orangehrmlive.com/web/index.php/auth/login"
    )
    print("OrangeHRM login page opened.")

    # 1. ID locator
    driver.find_element(By.ID, "app")
    print("ID Locator: Application container found")

    # 2. Name locator - username
    username = wait.until(
        EC.visibility_of_element_located((By.NAME, "username"))
    )
    username.send_keys("Admin")
    print("Name Locator: Username entered")

    # Name locator - password
    driver.find_element(By.NAME, "password").send_keys("admin123")
    print("Name Locator: Password entered")

    # 3. XPath locator - Login button
    driver.find_element(
        By.XPATH, "//button[@type='submit']"
    ).click()
    print("XPath Locator: Login button clicked")

    # 4. CSS Selector - Dashboard heading
    heading = wait.until(
        EC.visibility_of_element_located(
            (By.CSS_SELECTOR, "h6.oxd-topbar-header-breadcrumb-module")
        )
    )
    print("CSS Locator: Dashboard found")
    print("Dashboard Text:", heading.text)
    print("Page Title:", driver.title)

    if "dashboard" in driver.current_url.lower():
        print("Login Successful")
    else:
        print("Login Failed")

finally:
    driver.quit()
    print("Browser closed.")
    print("================================")
    print("PRACTICAL COMPLETED")
    print("================================")
```

Run command

```
python practical9_locators.py
```

Expected output (if login succeeds)

```
OrangeHRM login page opened.
ID Locator: Application container found
Name Locator: Username entered
Name Locator: Password entered
XPath Locator: Login button clicked
CSS Locator: Dashboard found
Dashboard Text: Dashboard
Page Title: OrangeHRM
Login Successful
Browser closed.
================================
PRACTICAL COMPLETED
================================
```

Result: The script demonstrates ID, Name, XPath and CSS Selector locators on the OrangeHRM demo website.

Practical 10 — Handle Alerts, Frames and Multiple Windows

Aim: To handle JavaScript alerts, switch into an iframe, and manage multiple browser windows using Selenium WebDriver.

Demo pages: Alerts • Frames • Browser Windows

Program: practical10_alerts_frames_windows.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 15)

try:
    driver.maximize_window()

    # PART 1: HANDLE ALERTS
    print("================================")
    print("PART 1: HANDLE ALERTS")
    print("================================")

    driver.get("https://demoqa.com/alerts")
    print("Alerts page opened.")

    wait.until(
        EC.element_to_be_clickable((By.ID, "alertButton"))
    ).click()

    alert = wait.until(EC.alert_is_present())
    print("Alert Message:", alert.text)
    alert.accept()
    print("Alert handled successfully.")

    # PART 2: HANDLE FRAME
    print("\n================================")
    print("PART 2: HANDLE FRAME")
    print("================================")

    driver.get("https://demoqa.com/frames")
    print("Frames page opened.")

    frame = wait.until(
        EC.presence_of_element_located((By.ID, "frame1"))
    )
    driver.switch_to.frame(frame)
    print("Successfully switched to iframe.")

    heading = wait.until(
        EC.visibility_of_element_located((By.ID, "sampleHeading"))
    )
    print("Text inside iframe:", heading.text)

    driver.switch_to.default_content()
    print("Successfully returned to main page.")

    # PART 3: MULTIPLE WINDOWS
    print("\n================================")
    print("PART 3: HANDLE MULTIPLE WINDOWS")
    print("================================")

    driver.get("https://demoqa.com/browser-windows")
    print("Browser Windows page opened.")

    parent_window = driver.current_window_handle
    print("Parent Window ID:", parent_window)

    wait.until(
        EC.element_to_be_clickable((By.ID, "windowButton"))
    ).click()

    wait.until(EC.number_of_windows_to_be(2))
    all_windows = driver.window_handles
    print("Total Windows:", len(all_windows))

    new_window = next(
        window for window in all_windows
        if window != parent_window
    )
    driver.switch_to.window(new_window)
    print("Switched to new window.")

    new_window_text = wait.until(
```

```
        EC.visibility_of_element_located((By.ID, "sampleHeading"))
    ).text
    print("Text in New Window:", new_window_text)

    driver.close()
    print("New window closed.")

    driver.switch_to.window(parent_window)
    print("Returned to parent window.")

    print("\n================================")
    print("PRACTICAL COMPLETED SUCCESSFULLY")
    print("================================")

finally:
    driver.quit()
    print("Browser closed.")
```

Run command

```
python practical10_alerts_frames_windows.py
```

Expected output (page text may vary)

```
PART 1: HANDLE ALERTS
Alerts page opened.
Alert Message: You clicked a button
Alert handled successfully.

PART 2: HANDLE FRAME
Frames page opened.
Successfully switched to iframe.
Text inside iframe: This is a sample page
Successfully returned to main page.

PART 3: MULTIPLE WINDOWS
Browser Windows page opened.
Total Windows: 2
Switched to new window.
Text in New Window: This is a sample page
New window closed.
Returned to parent window.

PRACTICAL COMPLETED SUCCESSFULLY
Browser closed.
```

Key methods: alert.accept() accepts an alert; driver.switch_to.frame() enters a frame; driver.switch_to.default_content() returns to the main page; driver.window_handles lists open windows; driver.switch_to.window() changes the active window; driver.close() closes the current window; driver.quit() closes the WebDriver session.

Result: The script demonstrates handling an alert, reading text inside an iframe, and switching between browser windows.

Troubleshooting and Final Checklist

Common errors

• **No module named selenium:** Run python -m pip install selenium in the VS Code terminal.

• **Python not recognised:** Reinstall Python with Add Python to PATH enabled, or try the py command.

• **ChromeDriver error:** Check Chrome is installed and internet access is available so Selenium Manager can obtain the driver.

• **Element not found or timeout:** The website may have changed, loaded slowly, shown a CAPTCHA, or displayed an overlay. Retry later and confirm the locator.

• **OrangeHRM login fails:** The public demo website or demo credentials may have changed; this is not necessarily a Python installation problem.

• **Browser closes after execution:** This is expected because the scripts call driver.quit() in a finally block.

Before submitting your practical

• Python and VS Code are installed.

• The Selenium package is installed.

• Each practical is saved in its own .py file.

• Each file is run separately from the VS Code terminal.

• Capture screenshots of browser and terminal output if your instructor requires evidence.

Safety note: These scripts open public demo websites and control Chrome. They do not intentionally modify Windows settings or personal files. Browser activity uses temporary CPU and memory, and the driver/package may use some disk space.